GitHub Actions y GitFlow

Guía práctica de trabajo en equipo

4.º del Grado en Ingeniería Informática · ULPGC

Programación de Aplicaciones Móviles Nativas (PAMN)

Material complementario de prácticas · 28 de octubre de 2026

Pregunta central: ¿cómo organizamos los cambios de nuestro proyecto y qué comprobamos antes de integrarlos?

Esta guía conecta la automatización ya presentada en clase con una estrategia de ramificación. Incluye un laboratorio Java, comandos Git, revisión mediante pull requests y actividades para acordar las reglas del proyecto de equipo.

Propuesta docente, sin carácter de normativa oficial ni de rúbrica de la asignatura. Las instrucciones del profesorado prevalecen.

1. Qué aprenderás y cómo usar la guía

Al terminar podrás crear una rama adecuada a cada tarea, proponer cambios mediante una pull request (PR), interpretar una ejecución de CI y preparar una versión con sus correcciones sincronizadas.

Recorrido recomendado (unas 3 horas, adaptable):

Fase	Tiempo	Resultado
Repaso y estrategia de ramas	25 min	Acuerdo inicial del equipo
Preparación y CI	30 min	Repositorio con pruebas automáticas
Feature, fallo y revisión	45 min	PR revisada hacia develop
Release y hotfix	50 min	Versiones y correcciones trazables
Debate y evidencias	30 min	Decisión razonada para el proyecto

Se presupone conocer `clone`, `add`, `commit`, `push` y el concepto de rama. Trabajad en parejas o equipos y alternad autoría y revisión.

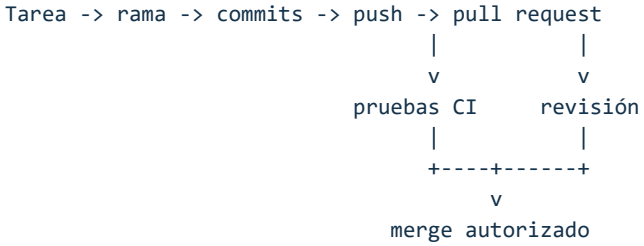
Material necesario: Git con `switch` (2.23 o posterior), cuenta de GitHub, editor, JDK 17 y Maven 3.9. Verifica `git --version`, `java -version` y `mvn -version`. La primera compilación necesita acceso a Maven Central. Las reglas de la organización pueden restringir Actions.

La carpeta `proyecto-ejemplo` contiene una aplicación de consola Java y siete pruebas JUnit. Usamos Java para continuar el ejemplo de la coordinadora. El objetivo es practicar el proceso; después podréis trasladarlo al sistema de construcción de vuestra aplicación móvil.

Relación con el material de clase. La página facilitada explica CI/CD, eventos, jobs, steps y ejemplos Java y Pages. Aquí se amplía la parte de colaboración y se aporta un repositorio reproducible. No es necesario desplegar en Pages para completar esta práctica. Material de la coordinadora.

2. De un cambio a una integración comprobada

Git registra versiones; GitHub aloja el repositorio y facilita la revisión. GitFlow es una convención de trabajo con ramas. GitHub Actions ejecuta automatizaciones definidas en YAML. Son piezas diferentes que se complementan.



CI (integración continua): construir y probar cambios con frecuencia. Las pruebas detectan regresiones conocidas; un resultado verde solo afirma que las comprobaciones configuradas han pasado.

Entrega continua: mantener el software preparado para publicar, pudiendo existir una aprobación manual. **Despliegue continuo:** publicar automáticamente los cambios que superan el proceso. Esta práctica implementa CI; ni un merge ni una etiqueta despliegan la aplicación por sí solos.

Concepto de Actions	En este laboratorio
Workflow	Archivo .github/workflows/ci.yml
Evento	push, pull_request o ejecución manual
Job	verificar, con nombre visible verificar-java
Runner	Máquina Linux temporal de GitHub
Step	Obtener código, preparar JDK, ejecutar Maven
Action	Pieza reutilizable invocada con uses
Comando	Orden ejecutada mediante run

Los steps de un job comparten su entorno de ejecución. Los jobs no comparten automáticamente el disco y pueden ejecutarse en paralelo; `needs` establece dependencias entre ellos. Nuestro único job evita esa complejidad inicial.

Lectura de apoyo: componentes de GitHub Actions.

3. GitFlow: responsabilidades de cada rama

El modelo clásico mantiene una rama estable y otra de integración. Aquí llamamos `main` a la rama que el artículo original denomina `master`. Los prefijos con barra son una convención del laboratorio; Git no les atribuye un comportamiento especial.

Rama	Se crea desde	Integra en	Propósito
main	Inicialización	-	Historial de versiones estables
develop	main al empezar	release	Integrar la próxima versión
feature/*	develop	develop	Una funcionalidad concreta
release/*	develop	main y develop	Estabilizar una versión
hotfix/*	main	main y develop	Corregir una versión publicada

En una release se permiten ajustes y correcciones, evitando nuevas funcionalidades. Si hay una release abierta al aparecer un hotfix, la corrección también debe llegar a esa release antes de publicarla; su cierre la propagará a `develop`. Puede adelantarse esa integración si hace falta.

```
Funcionalidad: develop -> feature/contar -> develop
Versión:      develop -> release/1.0.0 -> main [v1.0.0]
                         release/1.0.0 -> develop
Urgencia:     main -> hotfix/1.0.1 -> main [v1.0.1]
                      hotfix/1.0.1 -> develop
```

El esquema indica destinos de integración, no una secuencia literal de commits. La condición esencial es que una corrección publicada no se pierda al preparar la siguiente versión.

Adaptación a GitHub. Publicaremos las ramas temporales y haremos las integraciones con PR. Para este laboratorio elegimos merge commits, equivalentes en intención a `git merge --no-ff`: permiten reconocer las integraciones en el historial. No activéis una regla de historial lineal si vais a exigir estos merges.

No hace falta instalar la extensión `git flow`: practicaremos con Git estándar. Modelo original y reflexión de su autor.

4. Acordad la estrategia antes de crear ramas

El propósito de la práctica es tomar una decisión de equipo. Aprender GitFlow no obliga a adoptarlo en todos los proyectos.

Situación del proyecto	Alternativa a valorar
Entregas versionadas y estabilización separada	GitFlow
Equipo pequeño, una versión activa y cambios frecuentes	GitHub Flow
Integración muy frecuente, cambios pequeños y CI sólida	Desarrollo basado en trunk

GitHub Flow suele trabajar con `main`, ramas breves y PR. Trunk requiere especial disciplina para integrar cambios pequeños y mantener la rama principal saludable. Ninguna estrategia sustituye las pruebas ni la revisión.

Actividad de decisión (10 minutos). Escribid en `docs/estrategia-ramas.md`:

1. ¿Con qué frecuencia entregaremos? ¿Hay varias versiones en uso?
2. ¿Qué representa cada rama permanente y quién puede integrarla?
3. ¿Qué nombres, duración orientativa y destinos tendrán las ramas?
4. ¿Qué pruebas y revisiones son obligatorias antes de un merge?
5. ¿Cómo publicaremos versiones y propagaremos correcciones urgentes?
6. ¿Qué coste añade la estrategia y cuándo la revisaríamos?

Ejemplo de acuerdo para el laboratorio: usamos `main` y `develop`, una rama por tarea y PR pequeñas. Otro integrante revisa cada PR. El check `verificar-java` debe pasar. Releases y hotfixes se integran mediante merge commit en los destinos acordados. Eliminamos ramas temporales solo después de completar todos sus destinos.

Si vuestro proyecto elige GitHub Flow, documentad la justificación y adaptad el workflow para comprobar PR hacia `main`. Podéis completar primero el laboratorio de GitFlow en el repositorio de ejemplo para comprender el modelo completo.

Lecturas: GitHub Flow y reflexión del autor de GitFlow.

5. Preparar y ejecutar la aplicación

Descomprime `proyecto-ejemplo.zip`. Abre una terminal en la carpeta que contiene `pom.xml`. Los comandos de la guía funcionan en PowerShell y en shells habituales de Linux/macOS salvo indicación expresa. Sustituye los valores de ejemplo de usuario y repositorio antes de ejecutarlos.

```
mvn -B -ntp verify
java -cp target/classes es.ulpgc.pamn.Planificador "Estudiar Git" "Revisar PR"
```

Resultado esperado: Maven termina con `BUILD SUCCESS`, siete pruebas y cero fallos. La aplicación imprime dos tareas. Los datos solo viven en memoria; al cerrar el proceso se pierden.

```
proyecto-ejemplo/
  pom.xml
  README.md
  .gitignore
  .github/
    pull_request_template.md
    workflows/ci.yml
  src/main/java/es/ulpgc/pamn/Planificador.java
  src/test/java/es/ulpgc/pamn/PlanificadorTest.java
```

`agregar()` rechaza títulos nulos o en blanco y elimina espacios exteriores. `listar()` devuelve una copia inmutable. Las pruebas cubren lista vacía, orden, normalización, entradas inválidas y protección del estado interno.

Preparar el repositorio local, desde esa misma carpeta:

```
git init -b main
git config user.name "Tu nombre"
git config user.email "tu-correo-de-git@example.com"
git add .
git commit -m "chore: crear aplicación y CI inicial"
git status
```

La configuración anterior se limita a este repositorio. Usa tu identidad de Git y, si lo prefieres, el correo privado proporcionado por GitHub. `git status` debe indicar que no hay cambios pendientes. `.gitignore` evita incluir `target/` y ajustes del editor.

6. Publicar y proteger las ramas

En GitHub crea un repositorio llamado, por ejemplo, `pamn-gitflow-lab`. Elige la visibilidad acordada con el profesorado. Créalo vacío, sin README, licencia ni .gitignore adicionales, porque ya tienes el commit inicial. Copia su URL HTTPS y úsala en lugar de la siguiente URL de ejemplo:

```
git remote add origin https://github.com/TU-USUARIO/pamn-gitflow-lab.git
git push -u origin main
git switch -c develop
git push -u origin develop
```

La autenticación depende del cliente: sigue el inicio de sesión en el navegador del gestor de credenciales o usa GitHub CLI si ya lo tienes configurado. La contraseña de la cuenta no sirve como contraseña Git por HTTPS. No escribas tokens en el repositorio.

Abre la pestaña **Actions**, selecciona **CI Java** y comprueba que ha terminado correctamente. El envío inicial de `main` y el de `develop` generan ejecuciones independientes. Invita al resto del equipo mediante la configuración de colaboradores y que cada persona clone su copia del repositorio.

Después de una primera ejecución correcta, configura protección para `main` y `develop` en Settings, mediante reglas de protección de ramas o rulesets, según la interfaz disponible:

- Exigir una PR antes de integrar y al menos una aprobación de otra persona.
- Exigir el check `verificar-java`, seleccionándolo de los checks detectados.
- Exigir que la rama esté actualizada y que las conversaciones se resuelvan.
- Evitar force pushes y borrado de las ramas permanentes; revisar posibles permisos de bypass.
- Permitir merge commits y mantener desactivada la exigencia de historial lineal.

La disponibilidad de estas opciones depende del plan y de la visibilidad. Si no están disponibles, dejad constancia y aplicad revisión y comprobación manuales; una norma escrita no impide técnicamente el merge. En una prueba individual no podréis aprobar vuestra propia PR: la revisión requiere otra persona.

No activéis el borrado automático de ramas si puede eliminar una release o un hotfix antes de su segunda integración. Protección de ramas.

7. Entender el workflow que vais a ejecutar

El archivo del ejemplo contiene el siguiente workflow. La indentación YAML forma parte de la sintaxis: usa espacios.

```yaml
name: CI Java
on:
  push:
    branches: [main, develop, 'feature/**', 'release/**', 'hotfix/**']
  pull_request:
    branches: [main, develop, 'release/**']
  workflow_dispatch:
permissions:
  contents: read
jobs:
  verificar:
    name: verificar-java
    runs-on: ubuntu-latest
    timeout-minutes: 10
    steps:
      - name: Obtener el código
        uses: actions/checkout@v7
        with:
          persist-credentials: false
      - name: Preparar Java 17
        uses: actions/setup-java@v6
        with:
          distribution: temurin
          java-version: '17'
          cache: maven
      - name: Compilar y ejecutar pruebas
        run: mvn -B -ntp verify
```

`checkout` obtiene el código; el runner no lo tiene automáticamente. `setup-java` prepara Temurin 17 y la caché Maven. `verify` ejecuta las fases anteriores del ciclo Maven, incluidas pruebas y empaquetado: no hace falta añadir otro `mvn test` después. `-B` evita interacción y `-ntp` reduce mensajes de descarga.

Usamos versiones mayores de acciones para facilitar la lectura. En un proyecto real fijad un SHA completo revisado y planificad sus actualizaciones. `@v6` es la versión de la acción, mientras que `'17'` es la versión de Java. setup-java, checkout.

8. Cuándo se ejecuta y cómo diagnosticarlo

En `push`, el filtro se refiere a la rama que recibe el envío. En `pull_request`, se refiere a la rama de destino, llamada **base**. La rama que aporta los cambios se llama **compare** o head. Sintaxis oficial.

Acción	Ejecución esperada
Push a feature/contar-tareas sin PR	CI del push
Abrir PR de feature hacia develop	CI de la PR
Nuevo push a una feature con PR abierta	CI del push y de la PR
Merge de la PR en develop	CI del push en develop
Push de una etiqueta v1.0.0	Ninguna: no configuramos tags
Run workflow en Actions	Ejecución manual

La duplicación push/PR es deliberada para observar ambos eventos. En proyectos grandes podéis reducirla, manteniendo cobertura de los destinos y checks exigidos. Para que aparezca **Run workflow**, el archivo con `workflow_dispatch` debe estar en la rama predeterminada.

Investiga una ejecución: Actions > CI Java > ejecución > verificar-java > paso que falla. Confirma rama, evento y commit; después busca el primer error útil. Si falla una prueba, localiza su nombre y compara valor esperado y obtenido. Si falla al instalar Java, el problema todavía no está en las pruebas.

Seguridad mínima del ejemplo: el token automático tiene solo lectura de contenidos y checkout no conserva credenciales para pasos posteriores. No se necesitan secrets. Evita ejecutar código de PR no confiables con permisos elevados; no cambies este ejemplo a `pull_request_target` para sortear problemas de permisos.

Extensión opcional: conservar informes de Surefire como artefactos del workflow, incluso cuando fallen las pruebas. La caché acelera futuras ejecuciones; un artefacto conserva un resultado descargable. No confundas ninguno de ellos con publicar una release o desplegar.

9. Primera feature: trabajar y proponer cambios

Comprueba que tu copia de `develop` está actualizada y sin cambios pendientes:

```
git switch develop
git pull --ff-only origin develop
git switch -c feature/contar-tareas
```

Tarea: añade a `Planificador` el método `public int contar()`, que devuelve el número de tareas. Añade dos pruebas: cero al comenzar y dos después de agregar dos títulos. La implementación puede consultar el tamaño de la lista, sin exponerla.

```
mvn -B -ntp verify
git diff
git add src
git commit -m "feat: contar las tareas del planificador"
git push -u origin feature/contar-tareas
```

En GitHub abre **Pull requests > New pull request**. Selecciona **base: develop** y **compare: feature/contar-tareas**. Describe el comportamiento, las pruebas y cualquier decisión relevante. La plantilla incluida os ayuda a revisar lo necesario.

Experimento controlado: antes de integrar, cambia temporalmente la implementación de `contar()` para que devuelva siempre cero. Haz commit y push en esta misma feature. Las pruebas de dos tareas deben fallar y el check debe aparecer rojo. Guarda el enlace de esa ejecución; corrige la implementación, haz otro commit y push y espera al verde.

El objetivo es observar una regresión real detectada por las pruebas. No borres una prueba ni cambies su resultado esperado para ocultar el fallo. Tras añadir las dos pruebas propuestas, se esperan nueve pruebas en total.

Revisión por otra persona: comprobar que la base es correcta, el diff se entiende, hay pruebas útiles y no se han incluido archivos ajenos a la tarea. CI no conoce el acuerdo de ramas: con nuestro YAML, una feature dirigida por error a `main` también podría pasar. Revisar el destino sigue siendo responsabilidad del equipo.

10. Integrar, actualizar y resolver conflictos

Cuando haya aprobación y checks correctos, integra la PR con **Create a merge commit**. Puedes borrar la feature remota después del merge. Actualiza la copia local:

```
git switch develop
git pull --ff-only origin develop
git branch -d feature/contar-tareas
git fetch --prune
git log --graph --oneline --decorate --all
```

`--ff-only` evita que un pull cree un merge inesperado. Si falla, inspecciona el historial: puede haber commits locales que no están en remoto. No recurras a `reset --hard` ni a force push como solución automática.

Si develop avanza mientras trabajas en otra feature, incorpora sus cambios desde la propia feature. Este ejemplo supone que ya tienes abierta `feature/otra-tarea`:

```
git switch feature/otra-tarea
git fetch origin
git merge origin/develop
```

Si aparecen conflictos, `git status` indica los archivos afectados. Edita cada bloque marcado con `<<<<<<<`, `=======` y `>>>>>>>`, conserva el comportamiento correcto y elimina los marcadores. Después:

```
git add RUTA-DEL-ARCHIVO-RESUELTO
git commit -m "merge: resolver integración con develop"
mvn -B -ntp verify
git push
```

Sustituye la ruta por la real. Si decides cancelar esa integración antes de confirmarla, usa `git merge --abort`. No es necesario reescribir el historial de una rama compartida. El rebase es otra opción, pero exige un acuerdo específico si la rama ya está publicada.

Ejercicio opcional de conflicto: dos personas parten del mismo `develop` y cambian la misma línea de README en features distintas. Integrad la primera y actualizad la segunda. Explicad por qué Git pide una decisión humana aunque ambas versiones sean texto válido.

11. Preparar la primera release

Con la feature integrada, crea una rama de estabilización:

```
git switch develop
git pull --ff-only origin develop
git switch -c release/1.0.0
```

Edita únicamente la versión del proyecto en `pom.xml`: cambia `0.1.0-SNAPSHOT` por `1.0.0`. Crea `CHANGELOG.md` describiendo la aplicación y el contador. Ejecuta las pruebas y publica:

```
mvn -B -ntp verify
git add pom.xml CHANGELOG.md
git commit -m "chore: preparar versión 1.0.0"
git push -u origin release/1.0.0
```

1. Abre PR **base: main**, **compare: release/1.0.0**. Revisa el contenido completo de la versión. Espera aprobación y CI e integra mediante merge commit. Conserva la rama release.
2. Desde la copia actualizada de `main`, etiqueta el commit de integración. Antes de etiquetar, confirma que ese HEAD es el merge esperado y que su CI ha pasado.

```
git switch main
git pull --ff-only origin main
git log -1 --oneline
git tag -a v1.0.0 -m "Versión 1.0.0"
git push origin v1.0.0
```

3. Abre una segunda PR **base: develop**, **compare: release/1.0.0**. Así vuelven la versión y posibles correcciones de estabilización. Integra con revisión y CI. Solo entonces elimina la rama release remota.
4. Actualiza `develop`, elimina la rama release local con `git branch -d release/1.0.0` y ejecuta `git fetch --prune`.

Una etiqueta identifica un commit; una **GitHub Release** añade una ficha y notas sobre esa etiqueta; un despliegue ejecuta otro proceso. El workflow actual no hace estas dos últimas cosas. Para la próxima iteración podéis cambiar el POM a `1.1.0-SNAPSHOT` mediante una nueva PR hacia `develop`.

12. Corregir una versión: hotfix

Incidencia didáctica nueva: el producto decide que el mismo título, tras eliminar espacios exteriores, no debe repetirse. Consideradlo un defecto de la versión publicada. La comparación será exacta y sensible a mayúsculas; documentad esa decisión.

```
git switch main
git pull --ff-only origin main
git switch -c hotfix/1.0.1
```

Añade una prueba que agregue `Estudiar` y después `Estudiar` y espere `IllegalArgumentException` en la segunda llamada. Añade otra que confirme que un título diferente se acepta. Observa primero el fallo. Ajusta `agregar()` para comprobar duplicados sobre el título normalizado y vuelve a ejecutar las pruebas. Actualiza el POM a `1.0.1` y el changelog.

```
mvn -B -ntp verify
git add src pom.xml CHANGELOG.md
git commit -m "fix: rechazar tareas con título duplicado"
git push -u origin hotfix/1.0.1
```

Abre PR **hotfix/1.0.1 hacia main**. Tras revisión, CI y merge, actualiza `main`, comprueba el commit y su ejecución de CI y crea la etiqueta:

```
git switch main
git pull --ff-only origin main
git log -1 --oneline
git tag -a v1.0.1 -m "Corrección de títulos duplicados"
git push origin v1.0.1
```

Abre también PR **hotfix/1.0.1 hacia develop**. Integra la corrección antes de borrar la rama. Si `develop` tiene una versión `1.1.0-SNAPSHOT`, al resolver un conflicto conserva esa versión de desarrollo y el código corregido. Repite las pruebas después de resolverlo.

Si existe una release abierta, integra allí el hotfix y verifica que no vuelva a publicarse el defecto; documenta cómo llegará a `develop`. **Comprobación final:** la prueba de duplicados pasa tanto en la línea estable como en la de desarrollo. Un hotfix cerrado solo en `main` deja pendiente la propagación.

13. Problemas habituales y traslado a la app móvil

Síntoma	Qué revisar y hacer
No aparece ninguna ejecución	Ruta del YAML, commit enviado, filtros de rama y Actions habilitado
Check obligatorio siempre pendiente	Nombre del check y si el filtro excluye esa PR
Pasa en local, falla en GitHub	JDK, archivos no versionados y mayúsculas de rutas en Linux
Maven no encuentra dependencias	Acceso a Maven Central y configuración de red; repetir tras resolverla
Git pide contraseña por HTTPS	Inicio de sesión del gestor de credenciales o cliente configurado
Push rechazado	Permisos, rama protegida o remoto adelantado; inspeccionar antes de actuar
PR sin cambios	Base/compare incorrectas o commits todavía no enviados
Una prueba falla	Leer esperado/obtenido y corregir el comportamiento
No se puede fusionar	Revisiones, conflictos, CI pendiente o reglas del repositorio

Aplicación al proyecto Android/Kotlin. Mantened la lógica de ramas y reemplazad Maven por los comandos reales de vuestro proyecto. Si usáis Gradle Wrapper, versionad sus archivos y ejecutad en Linux `./gradlew testDebugUnitTest lintDebug assembleDebug`; en Windows el equivalente suele ser `gradlew.bat`. Verificad antes que esas tareas existen para vuestros módulos y variantes.

El JDK debe ser compatible con vuestro Android Gradle Plugin y Gradle Wrapper. Añadid la configuración del SDK y los permisos de ejecución que requiera el proyecto. Esta guía no incluye un workflow Android listo para todos los proyectos: la compilación JVM del ejemplo no comprueba manifiestos, recursos, emuladores ni firma de APK.

Separad las pruebas de la lógica de dominio de las instrumentadas, que necesitan dispositivo o emulador. La publicación en una tienda requiere firma, credenciales y otro flujo de aprobación. No incorporéis keystores, contraseñas o tokens al historial; cuando sean necesarios, configurad secrets y entornos con los permisos adecuados.

Antes de la siguiente práctica: ejecutar el comando de construcción de la app móvil en una copia limpia y después trasladarlo al runner. Así podréis identificar qué dependencias estaban instaladas solo en el equipo de una persona.

14. Evidencias, autoevaluación y referencias

Entrega propuesta para debatir en clase (adaptadla a las instrucciones del profesorado):

- Acuerdo de ramificación y justificación frente a una alternativa.
- Enlace a una PR de feature, con revisión de otro integrante.
- Enlaces al fallo controlado y a su corrección en Actions.
- Historial que muestre release, etiqueta y propagación a `develop`.
- Hotfix con prueba de regresión y evidencia de sus destinos de integración.
- Breve explicación de un conflicto o de una dificultad encontrada.

Preguntas de autoevaluación:

1. ¿Por qué una feature del modelo sale de `develop` y un hotfix de `main`?
2. ¿A qué rama se aplica `pull_request.branches`?
3. ¿Por qué un check verde puede acompañar a una PR con destino equivocado?
4. ¿Qué ocurre si borras una release antes de propagar sus ajustes?
5. ¿Qué diferencia hay entre tag, GitHub Release y despliegue?
6. ¿Qué coste evitaríais usando GitHub Flow en vuestro proyecto?

Claves breves: 1) se parte de integración o de la versión estable según el cambio; 2) a la base; 3) las pruebas no expresan todo el acuerdo; 4) queda pendiente llevar esos cambios a desarrollo; 5) referencia a commit, ficha de publicación y ejecución en un entorno; 6) menos ramas permanentes y sincronizaciones, si el contexto lo permite.

Fuentes y ampliación (consultadas el 28/10/2026):

- Material de PAMN: GitHub Actions. Punto de partida docente: CI/CD y ejemplos Java/Pages.
- GitHub: componentes de Actions. Terminología de automatización.
- GitHub: sintaxis de workflows. Eventos, filtros y permisos.
- Vincent Driessen: modelo de ramificación y reflexión posterior. Origen de GitFlow y elección según contexto.
- GitHub Flow. Alternativa con ramas breves y PR.
- GitHub: protección de ramas. Revisión y checks exigidos.
- actions/checkout y actions/setup-java. Acciones utilizadas en el ejemplo.

Los tiempos, actividades, aplicación, comandos de laboratorio y propuesta de evidencias son una elaboración didáctica para esta guía. Las interfaces y versiones de Actions pueden cambiar: revisad las fuentes antes de reutilizar el material en otro curso.